

Inter-Service Communication

Sync vs Async

Learning Objectives

After this section, students will be able to:

- Explain the difference between Synchronous and Asynchronous communication.
 - Decide when to use each communication style.
 - Understand the difference between Blocking and Non-Blocking execution.
 - Choose between OpenFeign, WebClient, and Kafka.
-

The Big Picture

In a Microservices architecture, services constantly communicate with each other.

Example:

Order Service



Inventory Service

Payment Service

Notification Service

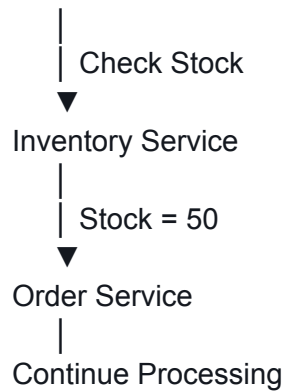
There are **two communication styles**:

1. Synchronous Communication
 2. Asynchronous Communication
-

1) Synchronous Communication (Sync)

The caller sends a request and **waits** for the response before continuing.

Order Service



Think of it like making a phone call.

You ask a question...

You wait...

Then continue.

Characteristics

- Immediate response
- Request/Response
- Caller waits
- Easier to understand
- Can increase latency

Typical Technologies

- OpenFeign
- WebClient
- RestTemplate (deprecated)

Example

Order Service

↓

Check Inventory

↓

Inventory replies



Create Order

Without the inventory response, the order cannot continue.

When should we use Sync?

Use Sync when the answer is required immediately.

Examples

✓ Check product stock

✓ Validate coupon

✓ Verify customer

✓ Charge payment

2) Asynchronous Communication (Async)

The caller sends a message and **does not wait**.

Order Service

Publish Event



Kafka



Notification Service



Send Email

Order Service continues immediately.

Think of it like sending an email.

You don't wait for the recipient to reply.

Characteristics

- No waiting
 - Better scalability
 - Loosely coupled
 - Event Driven
 - Eventually Consistent
-

Typical Technologies

- Kafka
 - RabbitMQ
 - Spring Cloud Bus
-

Example

Customer places an order

↓

Order Created

↓

Publish OrderPlaced Event

↓

Notification Service

↓

Send Email

Even if email takes 5 seconds...

The customer already received

Order Created Successfully

Sync vs Async

Feature	Sync	Async
Wait for response	✓ Yes	✗ No
Immediate result	✓ Yes	✗ No
Coupling	Higher	Lower
Performance	Lower	Higher
Complexity	Easier	More complex
Typical Tools	OpenFeign, WebClient	Kafka, RabbitMQ

Decision Framework

Ask yourself three questions.

Question 1

Do I need the answer immediately?

YES

↓

Use Sync

NO

↓

Use Async

Question 2

Can this operation happen later?

YES

↓

Async

NO

↓

Sync

Question 3

Is eventual consistency acceptable?

YES

↓

Async

NO

↓

Sync

Real Platform Examples

Scenario	Communication
Order → Inventory	Sync
Order → Payment	Sync
Order → Notification	Async

Order → Analytics Async

Order → Audit Log Async

Blocking vs Non-Blocking

Now comes another concept.

Many students confuse this with Sync/Async.

They are **NOT the same thing**.

Blocking

A thread waits until the response arrives.

Request

↓

Thread waits

↓

Response

↓

Continue

The thread is occupied during the waiting time.

Non-Blocking

The thread sends the request and is immediately released.

When the response arrives later, another callback continues the work.

Request

↓

Thread released

...

Response arrives

↓

Continue

No thread is blocked while waiting.

Important

Blocking / Non-Blocking is about **threads**.

Sync / Async is about **communication style**.

These are different concepts.

Four Possible Combinations

Communication	Thread Model	Possible?
Sync + Blocking	✓ Yes	Most common
Sync + Non-Blocking	✓ Yes	WebClient
Async + Blocking	⚠ Rare	
Async + Non-Blocking	✓ Kafka, Reactive	

OpenFeign

OpenFeign is the recommended tool for synchronous communication.

Order Service

↓

OpenFeign

↓

Inventory Service

↓

Return Response

Very simple.

Looks like calling a Java interface.

RestTemplate

Older HTTP client.

Requires writing every HTTP request manually.

Deprecated since Spring Framework 6.

Do not use it in new projects.

WebClient

WebClient is the modern HTTP client.

It supports:

- Synchronous communication
- Reactive programming
- Non-Blocking execution

Many developers think:

WebClient

↓

Async only

This is incorrect.

WebClient can be used synchronously.

Example

WebClient

↓

Inventory Service

↓

Wait for response

↓

Continue

It is still **Synchronous Communication**.

The difference is that the thread is **not blocked** while waiting.

CompletableFuture

CompletableFuture is a Java feature.

It allows asynchronous programming inside your application.

Call Service A

Call Service B

Call Service C

↓

Wait for all

↓

Continue

Useful when multiple independent calls can execute in parallel.

It is **not** a messaging system.

Kafka

Kafka is completely different.

Order Service

↓

Publish Event

↓

Kafka

↓

Inventory Service

↓

Notification Service

↓

Analytics Service

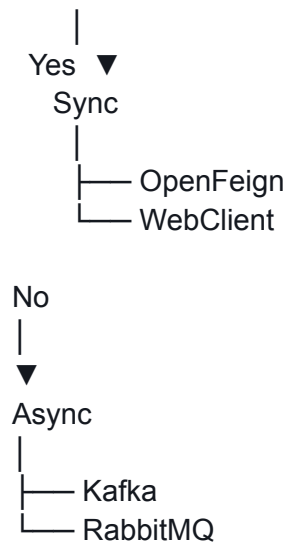
No service waits for another.

Which One Will We Use in This Course?

Session	Technology	Purpose
Session 6	OpenFeign	Synchronous communication
Session 7	Kafka	Asynchronous communication
Session 7	CompletableFuture	Parallel asynchronous tasks inside a service

Final Decision Tree

Need the response immediately?



Key Takeaways

- **Sync** means the caller waits for the response.
- **Async** means the caller continues without waiting.
- **Blocking** means a thread waits.
- **Non-Blocking** means the thread is released while waiting.
- **OpenFeign** is the recommended choice for synchronous service-to-service communication.
- **WebClient** is a modern non-blocking HTTP client and can be used for synchronous communication.
- **Kafka** is used for asynchronous, event-driven communication between services.
- **CompletableFuture** enables asynchronous execution within a single service and is not a replacement for Kafka.

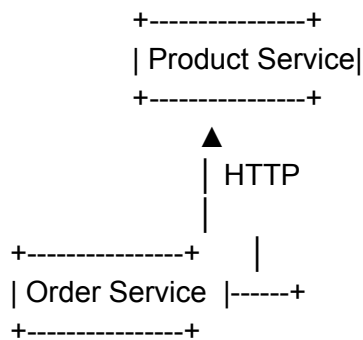
Inter-Service Communication

كيف تتواصل الـ Microservices مع بعضها؟

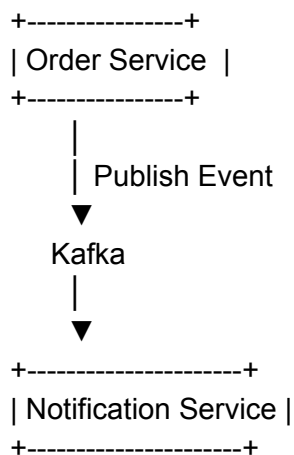
في نظام الـ Microservices كل Service مستقلة، لذلك لا يمكنها استدعاء الكود الموجود داخل Service أخرى مباشرة.

بدلاً من ذلك يتم التواصل عبر الشبكة (HTTP أو Messaging).

مثال:



أو

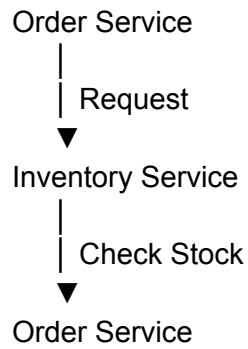


هناك طريقتان رئيسيتان للتواصل:

- Synchronous Communication
- Asynchronous Communication

أولاً: Synchronous Communication (Sync)

في الاتصال المتزامن، يقوم Service باستدعاء Service أخرى و ينتظر الرد قبل أن يكمل التنفيذ.



الفكرة تشبه الاتصال الهاتفي.

أنت تتصل...

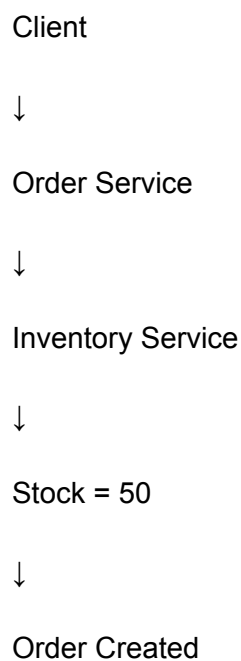
وتنتظر حتى يرد الطرف الآخر.

لا يمكنك إكمال العمل قبل الحصول على الإجابة.

مثال

يريد العميل إنشاء Order.

قبل إنشاء الطلب يجب معرفة هل المنتج متوفر أم لا.



هنا لا يمكن إنشاء الطلب قبل معرفة كمية المخزون.

إذن نستخدم **Sync**.

مميزات Sync

- ✓ سهل الفهم
- ✓ يعطي النتيجة مباشرة
- ✓ مناسب للعمليات التي تحتاج قرارًا فوريًا

العيوب

- إذا أصبحت Inventory بطيئة...
- فإن Order Service ستنظر.

Order



Waiting...



Waiting...



Inventory

كلما زاد الانتظار قل الأداء.

ولهذا نستخدم لاحقًا:

- Circuit Breaker
- Retry
- Timeout

ثانيًا: Asynchronous Communication ((Async

هنا لا ننتظر الرد.

بدلاً من ذلك نرسل Event ثم نكمل العمل مباشرة.

Order Service



Publish Event



Kafka



Notification Service

Order Service لا تعرف متى ستتم معالجة الرسالة.

ولا تنتظر.

مثال

بعد إنشاء Order نريد إرسال Email.

ليس من الضروري انتظار إرسال البريد.

Create Order



Publish OrderCreated Event



Return 201 Created



(بعد ثانية)

Notification Service



Send Email

مميزات Async

✓ أسرع

✓ لا يوجد Blocking

✓ الخدمات مستقلة عن بعضها

✓ يتحمل الضغط العالي

العيوب

لا تحصل على النتيجة مباشرة.

قد تصل الرسالة بعد ثانية أو دقيقتين.

مقارنة Sync vs Async

Async	Sync
لا ينتظر	ينتظر الرد
Non-Blocking	Blocking
Kafka / RabbitMQ	HTTP
نتيجة لاحقة	نتيجة فورية
أكثر تعقيداً	أبسط
يستخدم عندما يمكن الانتظار	يستخدم عندما نحتاج الإجابة الآن

كيف أختار؟

اسأل نفسك ثلاثة أسئلة.

السؤال الأول

هل أحتاج النتيجة الآن حتى أكمل؟

إذا كانت الإجابة نعم

Sync →

إذا كانت لا

Async →

السؤال الثاني

هل يمكن تأجيل العملية؟

مثلاً:

إرسال Email

يمكن تأجيله.

إذن Async.

السؤال الثالث

هل يمكن قبول Eventual Consistency؟

إذا كانت الإجابة نعم

Async →

إذا كانت لا

Sync →

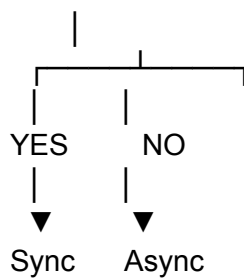
أمثلة من المنصة

النوع	العملية
Sync	Order → Inventory
Sync	Order → Product

Sync	Order → Payment
Async	Order → Notification
Async	Order → Analytics
Async	Order → Audit Log

Decision Framework

Need an answer immediately?



أدوات الاتصال المتزامن (Sync)

في Spring يوجد أكثر من طريقة لتنفيذ HTTP Call.

أشهرها:

- RestTemplate
- RestClient
- OpenFeign
- WebClient

لكن ليست كلها مناسبة لنفس الحالة.

مقارنة الأدوات

Status	Eureka Support	Reactive	Blocking	Style	Tool
Deprecated	يدوي	✗	✓	Imperative	RestTemplate

Recommended	يدوي	✗	✓	Fluent API	RestClient
Best for Microservices	تلقائي	✗	✓	Declarative	OpenFeign
Best for Reactive LoadBalancer	يدوي مع LoadBalancer	✓	✗	Reactive	WebClient

(قديم) RestTemplate (1)

كان هو HTTP Client التقليدي في Spring.

```
RestTemplate restTemplate = new RestTemplate();
```

```
Product product =
    restTemplate.getForObject(
        "http://localhost:8081/api/products/1",
        Product.class);
```

المميزات

- بسيط

العيوب

- Deprecated
- كتابة كثيرة
- لا يتكامل تلقائيًا مع Eureka

لا ننصح باستخدامه في مشاريع جديدة.

(البديل الحديث) RestClient (2)

ظهر في Spring Framework 6.1 ليكون البديل الرسمي لـ RestTemplate.

```
RestClient restClient = RestClient.create();
```

```
Product product = restClient.get()
    .uri("http://localhost:8081/api/products/1")
    .retrieve();
```

```
.body(Product.class);
```

لاحظ أن طريقة الكتابة أصبحت Fluent وأسهل.

المميزات

✓ API حديثة

✓ أسهل من RestTemplate

✓ مناسب لتطبيقات Spring MVC

العيوب

لا يزال عليك كتابة URL بنفسك.

مثلاً:

http://localhost:8081

أو استخدام Service Discovery يدوياً.

OpenFeign (الأفضل في Microservices)

بدلاً من كتابة HTTP Calls،

نكتب Interface فقط.

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping("/api/products/{id}")
    Product getProduct(@PathVariable Long id);

}
```

ثم نستخدمه مباشرة.

```
Product product = productClient.getProduct(id);
```

Spring يقوم بالباقي.

ماذا يفعل Feign؟

يقوم تلقائياً بـ

- Eureka عبر Service Discovery
- HTTP Call
- JSON Serialization
- Error Handling
- Load Balancing

بدون كتابة أي كود إضافي.

لماذا نحبه؟

تقريباً لا تكتب أي كود HTTP.

WebClient (4

إذا كان المشروع يستخدم Reactive (WebFlux)، فلا يمكن استخدام RestTemplate أو OpenFeign بالطريقة التقليدية.

هنا نستخدم WebClient.

```
WebClient webClient = WebClient.builder()
    .baseUrl("http://product-service")
    .build();
```

```
Mono<Product> product =
    webClient.get()
        .uri("/api/products/1")
        .retrieve()
        .bodyToMono(Product.class);
```

لاحظ أنه يرجع

Mono<Product>

وليس Product مباشرة.

لماذا؟

لأنه يعمل بطريقة Non-Blocking.

لا ينتظر الرد.

Blocking vs Non-Blocking

Blocking

Request

↓

Waiting...

↓

Waiting...

↓

Response

↓

Continue

الخيط (Thread) يظل مشغولاً طوال فترة الانتظار.

Non-Blocking

Request

↓

Continue doing other work

↓

Response arrives

↓

Handle response

لا يتم حجز الـ Thread أثناء الانتظار.

لذلك يستطيع نفس الـ Thread خدمة آلاف الطلبات.

(Reactive (WebFlux

Reactive يعتمد على:

- Mono
- Flux

بدلاً من إعادة Object مباشرة.

Mono<Product>

أو

Flux<Product>

ويحقق أداءً عاليًا عند وجود عدد كبير من الاتصالات المتزامنة.

متى أستخدم كل أداة؟

Need HTTP Call?

↓
▼

Is the application Reactive?

YES

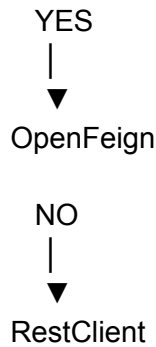
↓
▼

WebClient

NO

↓
▼

Is it Service-to-Service communication?



ماذا سنستخدم في هذه الدورة؟

السبب	Technology	Session
أفضل خيار للتواصل المتزامن بين الـ Microservices باستخدام Load Balancer و Eureka.	OpenFeign	Session 6
للتواصل غير المتزامن (Event-Driven).	Kafka	Session 7
لبناء خدمات Reactive عالية الأداء و Non-Blocking.	WebClient + WebFlux	Session 8
للتعريف بالبديل الحديث لـ RestTemplate في تطبيقات MVC Spring التقليدية.	RestClient	(اختياري – مقدمة)

الخلاصة

- إذا كنت تحتاج نتيجة فورية → استخدم **Synchronous Communication**.
- إذا كنت تستطيع إكمال العمل دون انتظار → استخدم **Asynchronous Communication**.
- في تطبيقات **Spring MVC** الجديدة، استخدم **RestClient** بدلاً من **RestTemplate** عند الحاجة إلى استدعاءات HTTP مباشرة.
- في **Microservices**، الخيار الافتراضي للتواصل المتزامن هو **OpenFeign** لأنه يتكامل تلقائياً مع **Eureka** و **Load Balancing** ويقلل كمية الكود.
- إذا كانت الخدمة **(Reactive WebFlux)**، فاستخدم **WebClient** لأنه يعمل بأسلوب **Non-Blocking** ويعيد **Mono** و **Flux** بدلاً من الكائنات التقليدية.